Backend Assignment - Simple Chat Application Using Redis

Develop a **Spring Boot application** that implements a simple chat application using **Redis** as the primary data store. The application should allow creating chat rooms, joining them, sending messages, and retrieving chat history. Redis will be used to store messages and manage real-time communication.

Requirements

1. **Chat Rooms**:
   - Users can create chat rooms with unique names.
   - Allow joining existing chat rooms.
2. **Messaging**:
   - Enable sending and retrieving messages in a chat room.
   - Store messages in Redis using a data structure like `Redis List`.
   - Retrieve the last `N` messages from a chat room.
3. **Real-time Messaging**:
   - Use **Redis Pub/Sub** for real-time message updates within chat rooms.
4. **Error Handling**:
   - Handle errors like:
     - Duplicate chat room names.
     - Sending messages to non-existent chat rooms.
5. **Persistence (Optional)**:
   - Use Redis persistence (e.g., snapshots or append-only file) to ensure durability.

API Specifications

1. Create a Chat Room

Endpoint:
`POST /api/chatapp/chatrooms`

Request Body:

```
{
  "roomName": "general"
}
```

Response:

```
{
  "message": "Chat room 'general' created successfully.",
  "roomId": "general",
  "status": "success"
}
```

---

2. Join a Chat Room

Endpoint:
`POST /api/chatapp/chatrooms/{roomId}/join`

Request URL:

bash
Copy code
`POST /api/chatapp/chatrooms/general/join`

Request Body:

```
{
  "participant": "guest_user"
}
```

Response:

```
{
  "message": "User 'guest_user' joined chat room 'general'.",
  "status": "success"
}
```

---

3. Send a Message

Endpoint:
`POST /api/chatapp/chatrooms/{roomId}/messages`

Request URL:

bash
Copy code
`POST /api/chatapp/chatrooms/general/messages`

Request Body:

```
{
  "participant": "guest_user",
  "message": "Hello, everyone!"
}
```

Response:

json
Copy code

```
{
  "message": "Message sent successfully.",
  "status": "success"
}
```

---

4. Retrieve Chat History

Endpoint:
`GET /api/chatapp/chatrooms/{roomId}/messages`

Request URL:

`GET /api/chatapp/chatrooms/general/messages?limit=10`

Response:

```
{
  "messages": [
    {
      "participant": "guest_user",
      "message": "Hello, everyone!",
      "timestamp": "2024-01-01T10:00:00Z"
    },
    {
      "participant": "another_user",
      "message": "Hi, guest_user!",
      "timestamp": "2024-01-01T10:01:00Z"
    }
  ]
}
```

Implementation Details

1. **Redis Data Structures**:
    - **Hash**: Store chat room metadata.
    - **Set**: Store participants in a chat room.
    - **List**: Store chat messages in chronological order.
    - **Pub/Sub**: Enable real-time message updates.
2. **Real-Time Messaging**:
    - Use Redis Pub/Sub to broadcast new messages to all participants in a chat room.
3. **Message Format**:

Each message in the Redis List should be a JSON string:
json
Copy code

```
{
  "participant": "guest_user",
  "message": "Hello, everyone!",
  "timestamp": "2024-01-01T10:00:00Z"
}
```

    -

Test Cases

Test Case 1: Create and Join Chat Room

- **Input**: Create a chat room `general` and join as `guest_user`.
- **Expected Output**: Room metadata is stored, and `guest_user` is added to the Redis Set.

Test Case 2: Send and Retrieve Messages

- **Input**: Send two messages in a chat room and retrieve the last message.
- **Expected Output**: Messages appear in the correct order.

Test Case 3: Real-Time Messaging

- **Input**: Publish a message to the room's Pub/Sub channel.
- **Expected Output**: All participants subscribed to the room receive the message.

Test Case 4: Error Handling

- **Input**: Attempt to send a message to a non-existent chat room.
- **Expected Output**: Error response indicating invalid room.

Test Case 5: Chat Room Deletion (Optional)

- **Input**: Delete a chat room.
- **Expected Output**: Redis keys related to the room are removed.

Evaluation Criteria

- **Functionality**:
    - Completeness of the specified APIs.
    - Proper use of Redis data structures.
- **Scalability**:
    - Efficient handling of concurrent participants and chat messages.
    - Efficient database operations
- **Error Handling**:
    - Robust validation for invalid inputs or unauthorized actions.
- **Code Quality**:
    - Code quality and structure (e.g., modular design, clear separation of concerns).
    - Will prefer classes properly refactored and following design patterns,
    - Clean, modular, and maintainable code with proper comments.
    - Will Prefer TDD, JUnit , **Test Coverage**: Comprehensive test cases covering various scenarios.
- **Real-Time Features**:
    - Effective use of Redis Pub/Sub for real-time updates.
- Assignment code submitted to be in **Java**
- REST Compliant APIs, (No UI)
- Testable by Postman, (No UI)
- Clarity of API documentation and how to run the project.